sim_1\new\PONG_BASYS3_TB.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;



entity PONG_BASYS3_TB is
end PONG_BASYS3_TB;

architecture PONG_BASYS3_TB_ARCH of PONG_BASYS3_TB is

constant ACTIVE: std_logic := '1';

component PONG_BASYS3
Port(
    clk: in std_logic;
    btnC: in std_logic;    --reset
    btnR: in std_logic;    --player one
    btnL: in std_logic;    --player two
    led: out std_logic_vector(15 downto 0);  --game board
    seg: out std_logic_vector(6 downto 0);   --score display (uncomment for advanced pong)
    an : out std_logic_vector(3 downto 0) --uncomment for advanced pong
);
end component;

signal clk:   std_logic;
signal btnC:  std_logic;
signal btnR:  std_logic;
signal btnL:  std_logic;
signal led:   std_logic_vector(15 downto 0);
signal seg:   std_logic_vector(6 downto 0);
signal an :   std_logic_vector(3 downto 0);

begin

process
begin
    clk <= ACTIVE;
    wait for 5ns;
```

```
    clk <= not ACTIVE;
    wait for 5ns;
end process;

UUT: PONG_BASYS3 port map(
    clk => clk,
    btnC => btnC,
    btnR => btnR,
    btnL => btnL,
    led => led,
    seg => seg,   --uncomment for advanced pong
    an => an      --uncomment for advanced pong
);

PONG_DRIVER: process
begin

    --Initialize to Zero
    btnC <= not ACTIVE;
    btnL <= not ACTIVE;
    btnR <= not ACTIVE;

    wait for 1ns;

    --Press reset
    btnC <= ACTIVE;
    wait for 1ns;
    btnC <= not ACTIVE;

    --Wait for when PLAYER_ONE state is set from reset
    wait until (led(15) = ACTIVE);

    --Start Game  (Player1)
    btnL <= ACTIVE;
    wait for 100ms;
    btnL <= not ACTIVE;

    ----------------------------------
    -- wait for ball movement
    wait until (led(0) = ACTIVE);
    ----------------------------------

```

```
    --Player 2's turn
    btnR <= ACTIVE;
    wait for 100ms;
    btnR <= not ACTIVE;

    ----------------------------------
    -- wait for a miss
    wait until (led(0) = ACTIVE);
    ----------------------------------

    --Player 2's turn
    btnR <= ACTIVE;
    wait for 100ms;
    btnR <= not ACTIVE;

    ----------------------------------
    -- wait for ball movement
    wait until (led(15) = ACTIVE);
    ----------------------------------

    --Player 1's turn
    btnL <= ACTIVE;
    wait for 100ms;
    btnL <= not ACTIVE;

    ----------------------------------
    -- wait for a miss
    wait until (led(15) = ACTIVE);
    ----------------------------------
wait;
end process PONG_DRIVER;

end PONG_BASYS3_TB_ARCH;
```